

AI Report

We predict this text is

Human Generated

AI Probability

23%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism



The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

Form3-Design.docx - 3/30/2026

KAFFAH ANNAS

Form3

Design

Title : Development of a Web-Based Mental Health Monitoring System with Mood Tracking for LightenUp

GROUP MEMBER:

No.

Student Name

Student ID

1. Vincent Julian Hadi Santoso

012202300018

2. Beri Yoel Apriyanto Fransisco Simanjuntak

012202300121

3. Kaffah An Nas

012202300064

Advisor : Abdul Ghofir, S.

Submitted for

Capstone Design Project

to Faculty of Computer Science

President University

TABLE OF CONTENT

TABLE OF CONTENT2

STATEMENT OF ORIGINALITY4

SCREENSHOT OF ZEROGPT5

PART 3DESIGN6

A. SYSTEM DESIGN7

1. System Architecture Overview7

2. Client Layer8

3. Load Balancer8

4. Web Server Cluster9

5. Controller Layer9

6. Business Logic Layer10

7. Data Access Layer10

8. Cache Layer (Redis)10

9. Database (LightenUpDB)11

10. Message Queue11

11. Worker Service12

12. External Service12

13. Interface Design12

14. Summary13

B HIERARCHICAL ITERATIVE DESIGN14

1. Block Diagram14

a. Block Diagram Level 014

b. Block Diagram Level 1 - System Overview15

c. Block Diagram Level 2 - Web Server Subsystem (ASP.NET Core MVC)18

d. Block Diagram Level 3 - Component & Function Level21

2. Global Variable / Message Parsing Interface23

- Mood Tracking & Questionnaire (Parameter Delivery)23

- Payment Processing (Message Delivery via REST)23

- Background Tasks (Message Queue)24

3. Scrum Methodology to Implement Steps in Software Engineering Design24

- Project Management Approach24

- Database Design25

UI UX Development25

- System Integration25

4. Component References & Libraries Used25

Hardware25

Software Frameworks26

- External Services26

- Database and Data Handling26

- Development and Utilities27

5. UML (Structure Diagrams, Behavioral Diagrams)27

- a. Entity Relationship Diagram27

- b. Use Case Diagram29

- c Activity Diagram31

- d Sequence Diagram34

- e Class Diagram35

C STANDARDS USED37

D. IMPLEMENTATION AND TESTING SCENARIO40

- 1. System Implementation Description40

- 2. Implementation and Testing Scenario Table42

REFERENCES57

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

Name: 1. Vincent Julian Hadi Santoso – 012202300018

2. Beri Yoel Apriyanto Fransisco Simanjuntak – 012202300121

3. Kaffah An Nas – 012202300064

Faculty: Computer Science

I hereby declare that my Capstone Design Project entitled "Development of a Web-Based Mental Health Monitoring System with Mood Tracking for LightenUp" is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, 7 March 2024

Signer 1

Signer 2

Signer 3

Vincent Julian Hadi Santoso – 012202300018

Beri Yoel Apriyanto Fransisco Simanjuntak – 012202300121

Kaffah An Nas – 012202300064

SCREENSHOT OF ZEROGPT

PART 3DESIGN

Consists of:

SYSTEM DESIGN

HIERARCHICAL / ITERATIVE DESIGN

STANDARDS USED

FUNCTIONAL TESTING

SYSTEM DESIGN

Figure 1 System Design

System Architecture Overview

The LightenUp platform is designed using a multi-layered architecture based on the ASP.NET MVC framework. This architecture separates the application into different functional layers in order to improve maintainability, scalability, and performance.

The system consists of the following major components:

Client Layer

Load Balancer

Web Server Cluster

Controller Layer

Business Logic Layer

Data Access Layer

Cache Layer

Database

Message Queue

Worker Service

External Services :

The architecture ensures that user requests are processed efficiently while allowing the system to scale horizontally when the number of users increases.

Client Layer

The Client Layer represents the users accessing the system through web-enabled devices such as:

Desktop Web Browsers

Mobile Browsers

Smartphones and Tablets

Users interact with the LightenUp platform through the web interface where they can.

Access the mood tracking interface

Fill in questionnaires

View results and analytics

Perform payment transactions if applicable

All communication between the client and the system occurs through HTTPS requests, ensuring secure data transmission.

Load Balancer

The Web Server Cluster hosts the ASP.NET MVC application that processes all incoming requests from users, which consists of multiple servers running the same application instance. This setup provides:

High Availability

Improved Performance

Fault Tolerance

If one Server fails, other Servers in the cluster continue handling user requests.

Web Server Cluster

The Web Server Cluster hosts the ASP.NET MVC application that processes all incoming requests from users, which consists of multiple servers running the same application instance. This setup provides:

High Availability

Improved Performance

Fault Tolerance

If one Server fails, other Servers in the cluster continue handling user requests.

Controller Layer

The Controller Layer is responsible for receiving HTTP requests from users and directing them to the appropriate components within the application.

HomeController handles general navigation and homepage requests.

MoodController manages mood tracking features.

QuestionnaireController handles submission and processing of user questionnaires.

PaymentController processes payment related operations.

Controllers act as intermediaries between the user interface and the business logic layer.

Business Logic Layer

The Business Logic Layer contains the core logic of the application and processes data according to the system's rules.

Major functionalities include:

Mood Analysis analyzes user mood data and generates insights.

Questionnaire Processing evaluates responses submitted by users.

Payment Processing handles payment validation and transaction logic.

This layer ensures that all application rules and workflows are executed correctly before interacting with the database.

Data Access Layer

The Data Access Layer manages communication between the application and the database. It is implemented using Entity Framework Core, which acts as an Object Relational Mapper to simplify database interactions. Key components include:

ApplicationDbContext – Handles database connections and entity management.

Database queries and CRUD operations.

This layer ensures that database operations remain separated from business logic.

Cache Layer (Redis)

The system includes a Redis Cache layer to improve performance and reduce database load. Redis stores frequently accessed data temporarily in memory, allowing the application to retrieve data faster than querying the database repeatedly. Examples of cached data include:

Frequently accessed questionnaire results

User session information

Mood analysis summaries

Database (LightenUpDB)

The main data storage component of the system is LightenUpDB, implemented using Microsoft SQL Server.

The database stores all persistent system data, including:

User accounts

Mood records

Questionnaire responses

Payment information

System logs

The database ensures data integrity and provides long-term storage for the application.

Message Queue

The system also incorporates a Message Queue to support asynchronous processing. Instead of executing time-consuming tasks immediately during user requests, these tasks are placed into the queue and processed separately. This improves the responsiveness of the application. Examples of queued tasks include:

Sending email notifications

Generating analytics reports

Processing third-party API requests

Worker Service

The Worker Service retrieves tasks from the message queue and processes them in the background. This service handles operations such as:

Email notifications

Analytics processing

Integration with third-party APIs

By offloading these tasks to a background worker, the main application remains responsive to users.

External Service

The LightenUp platform may integrate with several external services, including:

Email service providers

Payment Gateways

Analytics Services

These Services extend the system's functionality and allow the platform to interact with third-party systems when required.

Interface Design

The user interface of the LightenUp platform is designed to provide a simple and intuitive user experience. Key interface components include:

Home Page displays general information and navigation options.

Mood Tracking Interface allows users to log and track their emotional states.

Questionnaire Interface enables users to complete psychological or mood-related questionnaires.

Results Dashboard displays mood trends and insights.

Payment Interface allows users to complete subscription or service payments.

These interfaces are implemented using web technologies integrated with the ASP.NET MVC backend.

Summary

The LightenUp system design follows a scalable, modular, and layered architecture. By separating system responsibilities into distinct layers such as controllers, business logic, and data access, the system becomes easier to maintain and extend.

Additional components such as caching, message queues, and background workers improve system performance and ensure that the application remains responsive even under higher workloads.

This architecture provides a strong foundation for future expansion and integration with additional services.

HIERARCHICAL/ITERATIVE DESIGN

Block Diagram

Block Diagram Level 0

Figure 2.1.1 Block Diagram level 0

The LightenUp system interacts internally with three main actors: Employees (as patients), Psychologist, and the HR Department. In addition, it integrates with one external system, the Payment Gateway (DuitKu).Interface Information :

Communication between users and the web system is conducted over the HTTPS protocol to

● Sentences that are likely AI-generated.

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 10 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.